

**NOVA Information Management School**

Universidade Nova de Lisboa

**Asset-Centric Temporal Graphs for Crude-Oil  
Logistics:  
A Scalable, ML-Ready Schema**

*Master Thesis in Information Management Systems*

Pedro Porfirio

Student Number: 20241283

Supervisor: Professor Flavio Pinheiro



# Abstract

Crude-oil logistics data is not published at a single resolution and on a single source. Using EIA (Energy Information Administration) as an example, Inventories show up only at a handful of named hubs and at the strategic petroleum reserve sites while refinery throughput is reported by district. Most series are monthly; a few are weekly or daily.

A graph model built around the coarsest available level can no longer reason about routing. A model built around the finest level claims flows it cannot observe. This thesis addresses that mismatch, at the schema layer.

The schema is asset-centric. Wells, basins, gathering hubs, pipelines, terminals and refineries each appear as a node with its own inventory and its own mass balance. Topology is not declared on a separate table. Edges are relational variables on nodes, so the adjacency and the variable catalogue cannot fall out of step. The resolution hierarchy that connects a basin aggregate to its sub-basin children is read off the inputs of `sum_over_children` formula rows, not stored as a parent-child fact. Aggregates have no outflows of their own. A bidirectional pipeline is two outflow slots, never one signed flow. Balancing items, following the IEA convention, sit on every node to absorb measurement residuals.

Multi-resolution data is handled in a binding layer beneath the graph. The asset graph and the variable catalogue are persistent. What a scenario adds is, for each variable, the rule it follows: bind to a time series, derive from a formula, or hold at zero under the active resolution. A re-scope from basin level to PADD level edits the binding rows. The graph itself does not change.

The schema is realised as a Postgres database. The starter scenario covers the contiguous United States and Canadian crude inflow into PADD 2 and PADD 3, with 214 nodes, 395 directed flow edges and 1,646 variables. A four-stage notebook pipeline rebuilds the database end-to-end from a clean machine in about five to ten minutes. Chapter 4 reports the modelling outcomes for the United States case: scope, resulting graph, resolution choices and validation. Annex A documents the implementation in detail.

The application that motivated the framework was forecasting crude-oil inventories under a mean-reversion objective. That study is left to future work. What this thesis contributes is the representation. It states the schema, sets out the principles that keep it consistent under multi-resolution data, applies it to the United States case as a working instance, and gives a reference implementation that can be inspected and extended. A separate chapter argues for the schema's fit with downstream graph machine learning by tracing each design choice through to the input requirements of the standard vectorisation pipelines.

Keywords: graph schema, asset-centric modelling, temporal graphs, crude-oil logistics, multi-resolution data, mass balance, machine learning readiness, graph representation learning.

# 1. Introduction

## 1.1 Background and Motivation

Crude oil reaches a refinery only after a long sequence of physical handoffs. The first stage is production. In the United States, the Permian Basin in west Texas and southeast New Mexico is the largest single source: a tight-oil play exploited through horizontal drilling and hydraulic fracturing, with output of roughly 5.7 million barrels per day. The Bakken Shale in North Dakota and the Eagle Ford in south Texas operate on the same principle and add another 2.3 million barrels per day between them. Conventional onshore production continues in California, Oklahoma, Wyoming and Colorado, supplying older fields under decline. The federal offshore Gulf of America contributes about 1.8 million barrels per day from a mix of fixed platforms and floating production systems on deepwater discoveries. Alaska's North Slope, isolated by geography, adds a further 420 thousand barrels per day from Prudhoe Bay and connected fields.

From the wellhead, crude is collected by gathering systems, the small-diameter pipelines and battery sites that aggregate output across a producing region, and routed to origin terminals at the edge of the producing area. In the Permian, that means the gathering complexes around Crane, Midland and Wink. In the Bakken it means the Williston-Basin gathering blocks. In the Eagle Ford it means the Three Rivers terminal area. From the origin terminals, crude enters long-haul trunk pipelines: Gray Oak, Cactus II, Wink-to-Webster, BridgeTex and others moving Permian crude toward the Gulf Coast; DAPL carrying Bakken crude east through Patoka and on to Nederland; the Trans-Alaska Pipeline System (TAPS) carrying North Slope crude over the Brooks Range to the Valdez tanker terminal.

The trunk pipelines feed two kinds of destination. The first is storage. The Cushing hub in Oklahoma is the largest single example, with tank farms operated by Enbridge, Plains and Enterprise that collectively hold tens of millions of barrels and act as the physical settlement point for WTI futures contracts. The Gulf Coast carries its own set of storage hubs at St. James in Louisiana and at Beaumont and Houston in Texas. The four sites of the Strategic Petroleum Reserve (Bryan Mound, Big Hill, West Hackberry, Bayou Choctaw) hold the federal crude inventory in salt-cavern storage on the Gulf Coast. The second destination is onward transport: from a storage hub crude leaves by another pipeline (Seaway and MarketLink from Cushing to the Gulf, Capline from St. James to Patoka), by tanker loaded at Corpus Christi, Houston, Beaumont or the LOOP offshore terminal, or by short-haul rail or barge.

Refineries are where crude transforms. The Gulf Coast (PADD 3) carries the bulk of US refining capacity, with named operating sites in Houston, Port Arthur, Lake Charles, St. Charles and Baton Rouge. Midwest refineries at Whiting (Indiana), Wood River (Illinois) and Robinson (Illinois) take Canadian and Bakken crude through the Cushing and Patoka systems. East Coast refineries at Marcus Hook (Pennsylvania) and Bayway (New Jersey) and West Coast refineries around Anacortes, the Bay Area and Los Angeles round out the picture. Every refinery is designed for a particular range of crude qualities; running outside that range hurts product yield and processing efficiency. There is one corner-case destination outside the refinery network: a small number of thermoelectric power plants burn crude directly, mostly in the Caribbean and in island grids

where natural gas and refined product infrastructure is limited. These plants take the same physical product, but they bypass the refining transformation and go straight to combustion.

The reason this physical chain matters for modelling is in how it responds to disruption. Production and refining both operate close to their optimal rates as a matter of economics. Wells, once drilled, have low marginal cost and decline curves that make rapid down-cycles expensive; production is therefore run at or near full capacity subject to OPEC+ quotas, takeaway constraints and reservoir physics (Fattouh, 2011; Ribas, Hamacher and Street, 2010). Refineries are designed for specific throughput ranges, and operating outside those ranges reduces efficiency and degrades product yield (Fernandes, Relvas and Barbosa-Póvoa, 2013; Lima, Relvas and Barbosa-Póvoa, 2016). Switching crude grade is not a free decision either; it typically requires unit reconfiguration and planned downtime, with immediate revenue implications. The two ends of the chain are therefore inflexible on short horizons.

What absorbs the mismatch is inventory. When a hurricane forces an offshore platform to shut in for two weeks, refinery feedstock comes from storage. A pipeline rupture takes a trunk line out of service and downstream refineries draw on their tank farms. Tanker delays following Suez Canal congestion are bridged by regional storage at the importing refinery. Geopolitical shifts in trade flows, including sanctions on Iranian or Russian crude and OPEC+ quota changes, first show up as inventory swings at hubs and only later as production or refinery-run adjustments. Even the strategic petroleum reserve, which is reserved for severe disruption, operates on the same principle as commercial storage; it is a buffer at a different scale, not a replacement for the commercial inventory layer.

Inventories are therefore where the network's response to operational shocks shows up first. They sit on every storage-bearing node in the system: tank farms at Cushing and the Gulf Coast hubs, line and lease tankage at producing fields, refinery feedstock tanks, line fill in the long-haul pipelines, crude in transit on tankers and barges, and the SPR salt caverns. Forecasting any of those inventory variables is a network forecasting problem. What happens at one node is conditioned on what is flowing into it from upstream, what is leaving it downstream, and the persistent topology that determines who can flow to whom on what time scale.

The work available on this problem domain leans heavily on operations research and on per-node time-series forecasting. Operations research models, particularly linear and mixed-integer programming, enforce hard constraints and return optimal plans under their assumptions (Lima, Relvas and Barbosa-Póvoa, 2016; Kazemi and Szmerekovsky, 2015). Time-series methods such as ARIMA and VAR learn patterns from data, but in their standard form they are applied per node, with no explicit account of the network topology that connects nodes physically (Box et al., 2015; Hamilton, 1994). What is largely missing from the crude-oil logistics literature is a temporal-graph treatment in which the physical network is the load-bearing structure and the multivariate time series ride on top of it. The methodological tools exist in the broader graph representation learning and graph neural network literature (Wu et al., 2020; Jin et al., 2024), but the gap between those tools and the kind of representation a crude-oil logistics network needs has not been closed in published work. Closing it is what motivates this thesis. The remainder of the chapter sets out the problem in those terms, states the research objectives, fixes the scope, and outlines the structure that follows.

## 1.2 Problem Statement

The gap is representational rather than algorithmic. The methodological tools for joint space-time learning exist; the challenge is preparing a crude-oil logistics network so those tools can be used. Available graph representations of supply chains are typically committed to a single data resolution and declare edges as a separate object alongside nodes. That works when the data is uniform and topology can be audited externally. Crude-oil logistics has neither property. Production is reported by basin in some EIA series and by PADD in others. Inventories show up only at named hubs and the SPR. Refinery throughput is reported by district. Pipeline flows are reported per cross-border line on the Canadian side and only in aggregate elsewhere. Most series are monthly; a few are weekly or daily.

The schema this thesis develops is asset-centric, scalable and ready for downstream graph machine learning. Each of those words deserves to be taken in its simple sense.

Asset-centric means that every physical asset is a node with its own identity, attributes, inventory and mass balance. Regional aggregates exist as well, but they are observation views: they have no edges of their own. Flow lives on the physical layer.

Scalable means two things in practice. Adding a new asset is a small edit to the asset table and to the variable catalogue, with the topology following from the variables and the audit suite catching mistakes. Re-scoping the same asset graph to a different data resolution is an edit to a binding table, not a rebuild. The same graph supports basin-level, PADD-level and any other resolution that public data happens to admit.

Machine-learning ready means that the standard vectorisation pipelines can read the schema's outputs without further preprocessing of the schema-related kind. Adjacency is deterministic. Node features are uniform. Time alignment is explicit. Persistent topology and time-varying signals are kept apart. Chapter 5 makes this argument in detail by mapping each schema decision to the input requirements of a representative cross-section of vectorisation methods.

The schema we need has three properties that, taken together, none of the available representations enforce. First, edges have to follow from variables and not be declared on their own; otherwise the topology and the variable catalogue can fall out of step. Second, aggregates need to be views over physical nodes, with no edges of their own, so that observability and flow stay separate. Third, the per-scenario binding decisions about which level is authoritative for which subsystem need to live in their own layer rather than be baked into the graph build. None of these three properties is by itself novel. The contribution is to enforce all three at the same time, expose them as schema-level invariants, and demonstrate the result on a working implementation.

## 1.3 Research Objectives

The objectives of the thesis follow from the problem statement.

The first objective is to formulate a schema for crude-oil logistics graphs in which the asset is the unit of identity and the variable is the working primitive. Topology, aggregation and per-scenario scope are all expressed in terms of variables. No relationship table is added when a variable list already encodes the relationship.

The second objective is to enforce, as schema-level invariants, the design principles that keep the representation consistent under multi-resolution data. Edges fall out of variables. Aggregates are views. Balancing items remain on every node. Per-scenario node scope designates one level as authoritative and treats the others as inert. Bidirectional pipelines are paired slots. Each principle is mapped to the database constraint, trigger or index that defends it.

The third objective is to apply the schema to the United States crude-oil network as a working case study, end-to-end from a JSON contract to a loaded Postgres database. The instantiation has to produce inspectable artefacts (interactive maps, mass-balance audits, reachability matrices) that show the invariants holding on a concrete graph rather than only in the design.

The fourth objective is to argue for the resulting representation's fit with downstream graph machine learning. The argument is a fit analysis, not a method selection. Each schema decision is mapped to the input requirement it satisfies for at least one major family of graph vectorisation methods. The chapter that does this also names the residual preprocessing burden any concrete machine-learning pipeline would still have to handle.

## **1.4 Scope and Delimitations**

What this thesis does is define and implement a representation. It does not train or evaluate forecasting models. The original motivation, which was forecasting crude-oil inventories under a mean-reversion objective, is kept and discussed in Chapter 6 as the target application that shaped many of the design decisions, but the forecasting study itself is left to subsequent work.

The geographic scope of the seeded implementation is the contiguous United States plus Canadian crude inflow into PADD 2 and PADD 3. Mexican imports and offshore Gulf-of-America production are included where they affect PADD 3 routing. Alaska is included as a self-contained subsystem that routes through TAPS to West Coast refineries. Refined-product distribution downstream of refineries is out of scope; the graph stops at refinery transformation nodes. Thermoelectric direct-burn destinations are out of scope at the present time; they would attach as additional sink nodes if a future scope adds the relevant power-sector data sources.

The temporal scope is 2015 to 2025, monthly, matching the EIA series that drive the starter scope. The schema itself imposes no horizon. Weekly, daily or sub-daily scopes are admissible by changing the frequency on the scenario row and re-binding variables.

Refined-product yields, prices and economics are out of scope. Refineries appear as transformation nodes whose configuration attributes (Nelson Complexity Index, capacity, preferred crude slate) are recorded for future modelling, but the products that leave a refinery are not modelled as nodes here.

The machine-learning readiness claim is structural rather than empirical. The thesis does not benchmark GNN architectures against classical baselines; that benchmark is the explicit subject of future work. What it does is map schema decisions to vectorisation requirements, validate the mapping against a representative cross-section of the literature, and identify the residual preprocessing burden any downstream pipeline will face.

## 1.5 Thesis Structure

The remainder of the thesis is organised as follows.

Chapter 2 reviews graph representations of physical and supply-chain networks, locates the asset-centric and location-centric traditions within them, and condenses the graph representation learning literature into the subset that is relevant for assessing machine learning readiness.

Section 2.5 names the gap that the schema is designed to close.

Chapter 3 sets out the schema. It develops the asset-centric representation, the node and edge types, the temporal formulation, the inventory conservation constraint and the mathematical framework of node and edge definitions. The chapter ends with the post-build consistency checks that any concrete graph instance has to pass.

Chapter 4 reports the modelling outcomes for the United States case. The presentation is compact: scope, the resulting graph, the resolution choices made under the starter scope, and the validation outcomes from the audit suite. The full asset inventory and the descriptive detail are in Annex A.

Chapter 5 argues for the schema's fit with downstream graph machine learning. It maps each schema decision to a vectorisation requirement and walks through three method families to show fit and to name the residual preprocessing burden.

Chapter 6 sets out the future-work roadmap. It frames the original forecasting problem, names the candidate model families and describes the experimental setup that the schema enables but does not yet execute.

Chapter 7 concludes.

Annex A documents the implementation. It walks through the Postgres-backed asset graph, the four schema layers, the United States starter inventory in full, the consistency invariants in SQL, the audit suite and the artefacts the build produces.



## 2. Literature Review

This chapter reviews the work the schema builds on and contrasts itself with. The structure follows the working distinctions of the schema, which means it is not symmetric: some sections are kept brief, because their relevance to schema design is settled, and others go into more depth.

Section 2.1 covers graph representations of physical networks. This is where the asset-centric versus location-centric choice is made. Section 2.2 turns to asset graphs and supply-chain modelling more broadly, locating the contribution within that literature. Section 2.3 discusses network-level inventory conservation, the balance constraint that motivates the balancing-item-as-first-class design. Section 2.4 condenses graph representation learning into the subset that bears on machine-learning readiness; it is intentionally not a full GNN survey. Section 2.5 names the gap.

### 2.1 Graph Representations of Physical Networks

#### 2.1.1 Location-Centric vs Asset-Centric Representations

A first design decision in any graph-based logistics model is representational: what, exactly, should a node represent, and what should an edge represent? The literature commonly adopts one of two conventions. In practice, both can support learning, but they encode different assumptions about where the information (balance equations) resides (at locations versus at assets) and therefore differ in how naturally they can represent in-transit material, capacity constraints, and operational switching.

In a location-centric representation, nodes correspond to places (e.g., a refinery site, pipeline junction, or storage hub) and edges represent connections between places (e.g., pipeline segments or shipping lanes). This convention is widely used in traffic forecasting because sensor readings (speed, flow, occupancy) map cleanly to node features and the resulting graphs are compact (Li et al., 2018; Yu et al., 2018). The main limitation for commodity logistics is that material *in transit*; is not naturally a first-class state: vessel cargo and pipeline line-fill are typically absorbed into edge attributes, which makes it harder to reason about in-transit inventory as an operational variable.

In an asset-centric representation, each operational asset is a node—production fields, terminals, storage sites, pipeline segments, refineries, and (optionally) vessels—while edges encode feasible transfers between assets. This aligns closely with how constraints arise in practice: capacities, operating envelopes, and inventories belong to identifiable assets, not to abstract coordinates. A practical advantage is that the topology can remain fixed even when activity is intermittent: we define feasible edges once and set their flow features to zero when a route is inactive. This stable-topology design is convenient for many GNN architectures and, crucially for this thesis, it supports explicit modelling of inventories at both fixed storage nodes and mobile (in-transit) nodes.

#### 2.1.2 Application to Crude-Oil Logistics Networks

Crude-oil logistics is ultimately governed by asset capabilities (storage capacity, pipeline throughput, refinery complexity) and by how those assets are connected. For that reason, the

schema developed in this thesis is designed to be asset-centric. In the empirical implementation, however, the granularity we can instantiate is bounded by data availability: much of the public EIA reporting is regional (states and PADD districts). We therefore populate the schema at the level of assets that are consistently observable in those sources, while keeping the schema extensible so that finer-grained assets can be added when richer datasets become available.

Concretely, we represent reported regions as nodes where appropriate and connect them using major pipeline routes and seaborne corridors. The framework also accommodates more detailed representations—such as vessel nodes with AIS-derived features—but those are treated as extensions rather than as requirements for the results presented here. Chapter 3 formalises the node and edge types and explains how we preserve stable topology while allowing routes to become active or inactive over time.

This representational choice also highlights a recurring issue for learning-based models: historical flows describe what happened, but they may under-represent what is feasible. For example, a refinery's ability to switch crude slate is constrained by configuration (conversion units, desulphurisation capacity), not only by past purchasing patterns. Encoding such capabilities as static node attributes is one way to improve generalisation when the training window contains limited regime variation.

## **2.2 Asset Graphs and Supply-Chain Modelling**

### **2.2.1 Traditional Supply-Chain Optimization**

Supply-chain modelling has long used network abstractions in operations research. Facilities are represented as nodes and transportation links as edges with costs, capacities, and lead times. Mixed-integer linear programming (MILP) and related formulations support decisions such as facility location, capacity expansion, production planning, and inventory positioning across multiple echelons.

Within petroleum supply chains, optimisation models have addressed refinery planning, pipeline scheduling, and multimodal distribution. Fernandes, Relvas, and Barbosa-Póvoa (2013) formulate a deterministic MILP for strategic design of downstream petroleum networks. Kazemi and Szmerekovsky (2015) develop a multi-echelon, multimodal MILP for the U.S. downstream system, highlighting the value of representing multiple transport modes. Ghaithan, Attia, and Duffuaa (2017) propose a multi-objective model for tactical planning of integrated oil and gas chains. Ribas, Hamacher, and Street (2010) model uncertainty using stochastic and robust programming, and Lima, Relvas, and Barbosa-Póvoa (2016) review downstream oil supply-chain management and common modelling choices. These approaches provide strong constraint handling and interpretability, but their predictive performance depends on the quality of exogenous inputs (demand, availability, disruptions). This motivates complementary data-driven models that can learn patterns directly from historical observations.

### **2.2.2 Graph Neural Networks for Supply Chains**

GNN applications to supply-chain problems are recent but growing fast. Kosasih and Brintrup (2022) showed that GNNs could predict hidden links in supply-chain networks, uncovering latent dependencies that traditional methods miss. The SupplyGraph dataset, introduced by Wasi et al.

(2024), provided the first benchmark specifically designed for evaluating GNN-based supply-chain analytics, covering demand forecasting, production planning, and product classification. Evaluations on this benchmark have consistently shown that GNN-based models — particularly those incorporating temporal features — outperform traditional baselines including multilayer perceptrons and isolated LSTM models.

Several recent studies illustrate the potential of GNNs for supply-chain inventory optimisation. Alves and Mateus (2020) showed that modelling a multi-echelon supply chain as a graph and training a GNN on historical demand and lead-time data enables prediction of optimal inventory levels for each node, identification of bottleneck locations where stockouts are likely, and recommendations for corrective actions such as redistributing stock. Ahn, Song, Olivar, et al. (2024) developed a GNN-based probabilistic model for supply and inventory prediction using historical data from a global consumer goods company, showing that an attention-based graph architecture significantly improved prediction accuracy by capturing cascading effects across nodes. Dong and Samonte (2024) further confirmed the benefits of GNN-based optimisation for production supply chain networks. This network-wide perspective represents a fundamental advantage over traditional inventory models — economic order quantity or just-in-time approaches — that optimise individual nodes without considering their interdependencies (Alves and Mateus, 2020; Ahn et al., 2024).

## 2.3 Network-Level Inventory Conservation

At the network level, inventory conservation imposes a universal physical constraint on every node regardless of type — fixed assets such as terminals and refineries, and mobile assets such as vessels. For each node and each product, the change in inventory over any period must equal the difference between total inflows (production plus incoming shipments) and total outflows (consumption plus outgoing shipments). For a vessel node, this means the change in cargo level equals what was loaded at origin minus what was discharged at destination. Formally, for node  $i$  at time  $t$ :

$$I(i, t) = I(i, t-1) + P(i, t) + F_{in}(i, t) - C(i, t) - F_{out}(i, t)$$

where  $I$  denotes inventory,  $P$  denotes production,  $C$  denotes consumption, and  $F_{in}$  and  $F_{out}$  denote aggregate inflows and outflows respectively. Applied across all nodes and edges of the graph, this conservation equation creates a system of coupled constraints linking the temporal dynamics of individual nodes through the network structure. Any forecasting model targeting inventory levels must respect this physical constraint — either explicitly through architectural design or implicitly through training on data that already obeys it.

The mean-reversion objective follows directly: for each node  $i$ , the target inventory level  $I^*(i)$  equals the historical average inventory  $\bar{I}(i)$ , and the model should forecast the flow adjustments — changes in  $F_{in}$  and  $F_{out}$  for each edge — that would drive the system toward this target. The graph structure determines which flows can be adjusted; the temporal dynamics determine how fast adjustments propagate; the optimisation objective defines where the system should end up.

## 2.4 Graph Representation Learning and ML Readiness

This section condenses the graph representation learning literature into the subset that bears on whether a graph schema is fit for downstream machine learning. The condensation is intentional: a full GNN survey is not the contribution of this thesis. Chapter 5 returns to specific method families to validate the fit between the schema and standard vectorisation pipelines.

### 2.4.1 Foundations of Network Embedding

Graph representation learning maps nodes (and sometimes edges) into vectors that retain information useful for a downstream task. Early approaches were largely spectral, using eigenvectors of Laplacian operators to produce low-dimensional embeddings. While theoretically grounded, spectral methods scale poorly and are inconvenient when graphs evolve. This motivated embedding methods trained with stochastic optimisation that scale to larger networks and can be updated more easily.

DeepWalk (Perozzi, Al-Rfou, and Skiena, 2014) introduced a simple and scalable idea: treat short random walks on a graph as sentences; and learn node embeddings using the same Skip-gram objective used in word embeddings. Nodes that frequently co-occur in walks—because they are close in the topology—are placed near one another in the embedding space. The method is attractive as a lightweight structural baseline and as a practical way to encode connectivity without explicit matrix factorisation.

Node2vec (Grover and Leskovec, 2016) generalised DeepWalk by biasing random walks to emphasise either local neighbourhoods or broader community structure, controlled by its return and in-out parameters. LINE (Tang et al., 2015) takes a different route by optimising objectives that preserve first- and second-order proximity directly, which can be efficient on very large graphs. These methods provide useful descriptors of topology, but they are fundamentally static: by construction, they do not model how node states (flows, inventories) evolve through time.

In this thesis, the feasible connectivity of the logistics system is treated as largely stable over the study period, so structural embeddings are sensible as topology-only baselines. However, the forecasting objective depends on time-varying quantities—production, flows, and inventories. A topology-only embedding cannot, on its own, represent inventory build and draw patterns or changing utilisation. This motivates models that incorporate node and edge features (spatial GNNs) and, ultimately, models that incorporate time directly (temporal graph methods and hybrid spatio-temporal architectures).

A second limitation is important in crude-oil applications: embeddings trained purely on observed interactions tend to encode realised behaviour rather than physical capability. If a refinery rarely switches crude slate during the training window, a flow-driven embedding may implicitly treat alternative supply routes as irrelevant, even if the refinery is technically able to process those grades. This motivates augmenting time-varying operational signals with static configuration attributes so that the representation reflects feasible operating regimes.

Inductive GNNs (e.g., GraphSAGE) can improve generalisation by learning aggregation rules that transfer across similar neighbourhood structures, but they still benefit from diverse historical regimes. In this thesis, we additionally encode capability directly via static attributes—most

importantly refinery configuration features such as conversion unit presence, Nelson Complexity Index, and crude-quality tolerance. These attributes provide a grounded basis for learning similarities between assets even when their observed flow histories differ, and they support modelling of rare but feasible transitions such as feedstock switching. Chapter 3 specifies how these configuration features are incorporated into node representations.

### 2.4.2 Graph Neural Networks

Graph neural networks (GNNs) learn node representations by combining node features with information from neighbouring nodes via message passing (Gilmer et al., 2017). At each layer, a node updates its representation by aggregating transformed messages from its neighbours (and, in many variants, from itself). Stacking layers allows information to propagate across multiple hops, producing embeddings that reflect both local state and network context.

The Graph Convolutional Network (GCN) of Kipf and Welling (2017) is a standard baseline that aggregates neighbour information using a normalised adjacency operator. In our setting, applying a GCN to each time snapshot yields a spatially enriched embedding: inventories and flows at a node are combined with upstream and downstream signals, reflecting the network coupling implied by the physical system. GCN is therefore a useful reference point for evaluating whether more expressive aggregation mechanisms are needed.

Graph Attention Networks (GAT) (Veličković et al., 2018) replace uniform aggregation with learned attention weights, allowing a node to prioritise the neighbours that are most informative for the task. This is appealing for logistics graphs because connections differ materially (e.g., high-capacity pipelines versus minor routes) and because relevance can change with regime (e.g., a backup supply route becoming important during a disruption).

GraphSAGE (Hamilton, Ying, and Leskovec, 2017) learns neighbourhood aggregation functions that can be applied inductively, enabling embeddings for nodes not seen during training. This matters when the network is updated (assets are added or reclassified) and we wish to embed new nodes without retraining the entire model. Neighbour sampling also improves scalability by limiting the number of messages processed per node.

We use survey taxonomies to organise model choices and evaluation. Wu et al. (2020) provide a broad overview of spectral and spatial GNN families, while Jin et al. (2024) focus on how GNNs are adapted for time-series problems such as forecasting and imputation. These references inform the method families compared in Chapter 4 and the forecasting architectures evaluated in Chapter 5.

Temporal extensions, including discrete-time snapshot models, continuous-time event-based models, and hybrid temporal-graph architectures, are reviewed only at the depth needed to support the readiness argument in Chapter 5. The point that matters for schema design is that all three families assume a clean separation between the persistent topology of the graph and the time-varying signals on its nodes and edges. The schema we propose makes that separation explicit at the database layer.

## 2.5 Synthesis and Research Gap

The literature on graph representations of supply chains, on energy balance accounting and on graph representation learning addresses parts of the problem this thesis works on. Asset-centric modelling is well established for transmission networks but is less common for energy commodity supply chains, where location-centric representations dominate. Mass-balance accounting with explicit balancing items is standard in IEA energy statistics. Graph representation learning gives a rich set of vectorisation techniques. The literature is, for the most part, silent on what it takes to prepare a real, multi-resolution logistics network so that those techniques are applicable.

The gap the schema closes is therefore representational rather than algorithmic. A schema is needed that commits to asset-centricity without losing the ability to talk about regional aggregates; admits multi-resolution observation without rebuilding the graph; keeps balance accounting on every node without forcing premature imputation; and presents its outputs in a form the standard vectorisation pipelines can read. Each property has antecedents. Enforcing them together as schema-level invariants, and showing that the result holds on a working implementation, is the contribution. The remaining chapters develop and defend it.

## 3. Graph Schema Design for Crude-Oil Logistics Networks

### 3.1 Overview of the Graph Representation

Applying graph-based methods to a logistics network requires an explicit schema: which asset types appear as nodes, which pathways appear as edges, and which attributes are carried by each. The schema must be rich enough to preserve the operational logic of the crude-oil system (flows, capacities, delays, inventories) while remaining simple enough to support scalable learning and forecasting. In this chapter we define a directed, weighted graph representation at the regional level, aligned with the granularity of the available data, and we make the design decisions needed to keep the topology stable through time.

#### 3.1.1 Physical Flow Structure of Crude-Oil Networks

Before formalising the schema, it helps to trace the physical journey of crude oil through the supply chain, since the graph structure is ultimately a faithful abstraction of these material flows.

Crude oil originates at three principal categories of production asset: onshore wells, offshore platforms (fixed rigs and FPSOs), and unconventional shale or tight-oil formations exploited through hydraulic fracturing. These assets differ in production profile and logistical constraints but share the same role in the graph — source nodes that generate flow with no upstream crude inflow.

From the gathering hub, crude moves via one of three transport modes. Pipelines dominate in most mature producing regions, carrying crude continuously through trunk and feeder lines at low marginal cost over long distances. Where pipeline infrastructure does not reach — or where cross-ocean movements are required — maritime tankers take over, ranging from VLCCs on long-haul routes to Aframax and Suezmax vessels on regional trades. Rail and truck transport plays a supplementary role, primarily for short-haul movements and in regions where pipeline access is limited, such as the Permian Basin during periods of rapid capacity growth.

A fundamental routing decision occurs at this midstream stage — one central to the inventory dynamics this thesis models. Crude can flow directly to a refinery, bypassing intermediate storage, or it can be injected into a storage terminal or strategic petroleum reserve (SPR) first. The direct route is preferred when refinery demand is strong and transportation is tightly coordinated, minimising holding costs. The storage route is preferred when supply exceeds immediate refinery demand, when prices favour contango strategies, or when operators are building buffer stocks against anticipated disruptions. This switching behaviour — and its effect on inventory levels at storage nodes — is precisely what drives the mean-reversion dynamics the model is designed to forecast.

Crude is released from storage to refineries as operational requirements dictate. The refinery is the transformation node of the network: it converts raw crude into a slate of refined products through distillation, fluid catalytic cracking (FCC), reforming, and other processes. Gasoline, diesel and jet fuel, LPG, naphtha and petrochemical feedstocks, and heavy fuel oil then flow onward to their respective end-use markets.

The topology shown in Figure 3.1 is not a simple linear chain — several structural features have direct consequences for the graph model. The gathering hub is a convergence point whose state depends on the combined output of multiple upstream nodes. The three transport modes create parallel paths between the same origin-destination pairs, with different transit times, capacities, and costs; disrupting one mode redistributes flow onto the others. The optional storage layer introduces time-buffering: crude entering a terminal at time  $t$  may not reach a refinery until time  $t + \tau$ , where  $\tau$  depends on operational priorities and price signals. The refinery itself drives demand backwards through the network — a turnaround or capacity change propagates upstream as reduced offtake, affecting storage levels and ultimately upstream production scheduling.

A graph naturally encodes convergence through node fan-in, parallelism through multiple edges between node pairs, time-buffering through edge transit-time attributes, and bidirectional influence through the GNN message-passing mechanism.

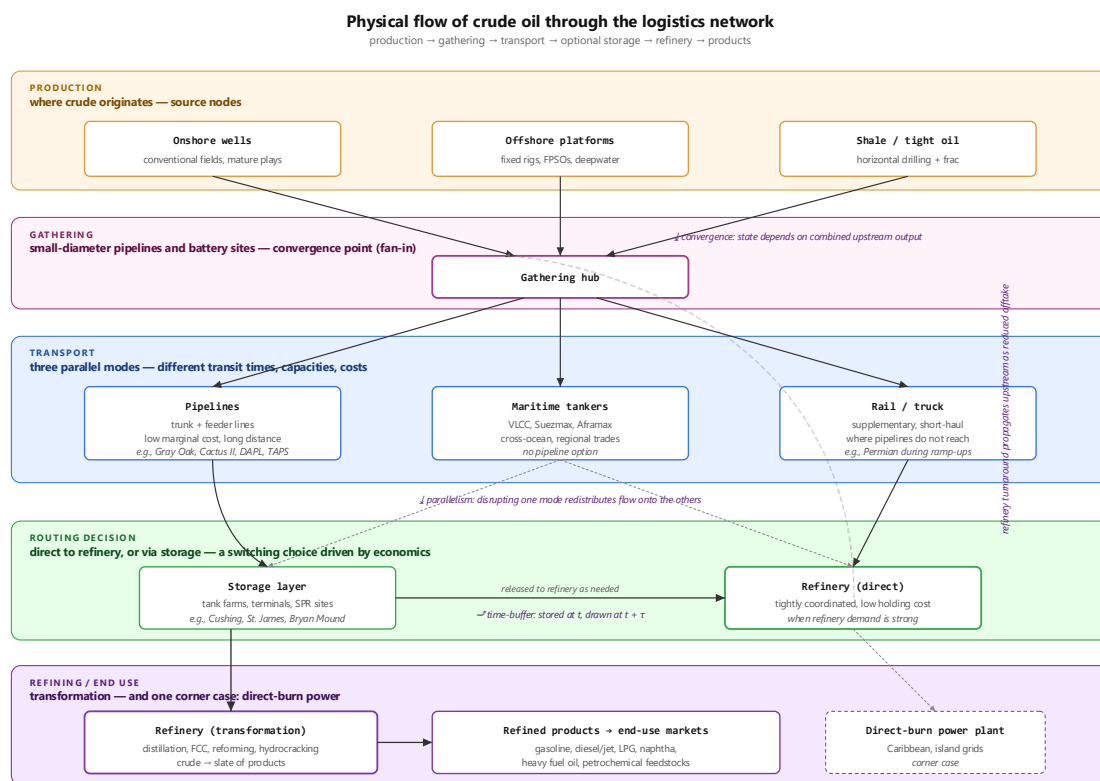


Figure 3.1 — Physical flow of crude oil through the logistics network, from production to refinery (or, in the corner case, direct-burn power generation). Source: Code/asset\_graph/physical\_flow.svg.

### 3.1.2 From Physical Network to Graph Schema

The logistics network is modelled as a directed, weighted graph  $G = (V, E)$ , where nodes correspond to operational assets and edges encode the physical or contractual connections among them. The node set  $V$  is partitioned into five functional types—production, storage, transit, refinery, and consumption—each characterised by a common set of time-varying variables. Edges carry flow rates, transit times, and capacity utilisation. The full schema is detailed in the sections that follow.



### 3.1.3 Refinery Processes and the Transformation Node

The refinery occupies a unique role in the logistics network as the sole transformation node — consuming crude oil as input and producing an entirely different set of commodity streams as output. Understanding refinery economics and process architecture is essential for interpreting the inventory dynamics modelled here, since the composition and intensity of refinery demand is a primary driver of crude drawdowns from storage.

Refinery processing proceeds in three successive layers. Atmospheric distillation comes first: crude feedstock is heated to approximately 350°C and fed into a fractionation column that separates it into discrete fractions by boiling point — refinery gas and LPG at the top (below 40°C, C1-C4); naphtha (40-180°C, C5-C10); kerosene and jet fuel (150-250°C, C10-C16); diesel and atmospheric gas oil (200-350°C, C14-C20); heavy gas oil (300-400°C, C20-C40); and atmospheric residue (above 400°C, C40+). Distillation alone yields commercially usable products from roughly 40-50 per cent of a typical crude barrel, leaving a substantial fraction of heavy material that requires further processing.

The second layer consists of conversion units that crack large hydrocarbon molecules from the heavy fractions into smaller, higher-value products. Three principal conversion processes are used in modern complex refineries. Fluid Catalytic Cracking (FCC) vaporises heavy gas oil and contacts it with a hot powdered zeolite catalyst at approximately 500°C, breaking long carbon chains into gasoline-range molecules within seconds; the catalyst is continuously regenerated by burning off deposited coke. FCC is the primary reason a complex refinery can achieve gasoline yields exceeding 50 per cent of crude input. Hydrocracking operates under high hydrogen pressure to convert vacuum gas oil into ultra-low-sulphur diesel and jet fuel — at higher capital and hydrogen cost, but with superior product quality and flexibility. Delayed coking thermally cracks the vacuum residue into distillates and solid petroleum coke, recovering value from material that would otherwise have no product outlet. The choice of conversion configuration (FCC-dominant, hydrocracker-dominant, or coker-equipped) determines a refinery's yield slate and, consequently, its appetite for specific crude grades: FCC-equipped refineries favour lighter, lower-sulphur feedstocks, while hydrocracker-equipped refineries can efficiently process heavier sour crudes.

The third layer is treating and blending. Hydrotreating units use hydrogen to remove sulphur from distillate streams, producing hydrogen sulphide that is subsequently converted to elemental sulphur in a Claus unit. The catalytic reformer restructures straight-run naphtha to increase its octane number, producing both high-octane gasoline blendstock and the hydrogen consumed by hydrotreating and hydrocracking units elsewhere in the refinery. Final blending combines intermediate streams to meet the precise specifications — octane rating, cetane index, flash point, viscosity, cloud point — required by product markets and regulatory standards.

For the graph model, the practical implication is that the refinery node is characterised not only by its throughput rate but also by its configuration. EIA refinery yield data — barrels of gasoline, distillate, jet fuel, and residual fuel produced per barrel of crude input — directly reflects which conversion units a refinery operates and at what intensity. A refinery with a large FCC but no hydrocracker produces a gasoline-heavy yield slate; one with a hydrocracker skews toward diesel and jet fuel. These configuration differences are observable in the PADD-level production and

input data used in this thesis and constitute one source of node heterogeneity in the graph representation.

### **3.1.4 Maritime Transport: Tanker Classes, Voyage Operations, and Global Trade Routes**

Maritime tanker shipping is the principal mechanism for intercontinental crude oil trade. Unlike pipelines — fixed infrastructure with defined origin-destination pairs — tankers are mobile capital assets whose routing decisions are made dynamically in response to price signals, freight rates, and port availability. This mobility introduces a layer of operational flexibility, and corresponding complexity, that is consequential for the inventory dynamics the graph model captures.

Crude tankers are classified by deadweight tonnage (DWT), and size class matters operationally because it determines which ports, canals, and offshore terminals a vessel can physically access. Ultra Large Crude Carriers (ULCCs, above 320,000 DWT) and Very Large Crude Carriers (VLCCs, 200,000–320,000 DWT, roughly 2 million barrels capacity) operate on long intercontinental routes and often discharge via offshore moorings or lightering because loaded draught can exceed 20 metres. Suezmax vessels (120,000–200,000 DWT, roughly 1 million barrels) are sized to transit the Suez Canal at its draught limits and dominate many Atlantic Basin trades. Aframax tankers (80,000–120,000 DWT) typically serve regional trades (Caribbean, North Sea, Baltic, and the U.S. Gulf Coast). Smaller Panamax/LR1 vessels (55,000–80,000 DWT) are constrained by canal dimensions, while Medium Range and Handysize vessels (below 55,000 DWT) handle coastal movements and refined-product distribution.

A crude tanker voyage can be described in four phases: (1) ballast voyage to the load port, (2) loading, (3) laden transit, and (4) discharge. Loading and discharge typically last 24–48 hours, while intercontinental passages can last several weeks. Ship-to-ship (STS) transfer and lightering are particularly important for large crude carriers because many ports cannot accommodate their loaded draught; in these operations, cargo is transferred offshore to smaller vessels that can berth at conventional terminals. In a future extension using AIS-derived signals, STS activity can be proxied by co-location and low-speed behaviour of vessel pairs over extended periods, potentially combined with draught changes.

Global crude trade is organised around a small number of dominant corridors, and several geographic chokepoints create shared risk across many routes. The Arabian Gulf–Asia corridor passes through the Strait of Hormuz and often through the Strait of Malacca; disruptions or heightened risk premia at either chokepoint can reshape voyage times and routing. The Arabian Gulf–Europe route introduces a trade-off between canal fees and sailing distance: Suezmax vessels may transit the Suez Canal directly, while VLCCs often route around the Cape of Good Hope depending on economics and operational conditions. West African crude frequently moves on Suezmax vessels to European and U.S. destinations. Since the removal of the U.S. crude export ban (December 2015), the U.S. Gulf Coast has simultaneously acted as a major import destination for heavier grades and a growing export origin for light sweet crude, reflecting grade-based arbitrage rather than logistical inefficiency. These corridor effects matter for our graph representation because they imply correlated delays and disruptions across multiple edges rather than isolated, independent shocks.

In graph terms, maritime shipping lanes are directed edges with transit-time attributes reflecting voyage durations, capacity attributes reflecting vessel class constraints, and flow attributes drawn

from AIS-derived cargo estimates and EIA import records. The chokepoints — Hormuz, Suez, Malacca, the Cape — represent latent risk factors that can simultaneously affect multiple edges in the graph, producing correlated disruptions across otherwise distant node pairs. Capturing these correlations is one of the structural advantages that graph-based representations hold over univariate time-series approaches.

### 3.2 Node Types and Attributes

The logistics network is modelled as a directed, weighted graph  $G = (V, E)$ , where every physical asset is represented as a node. The node set  $V$  is partitioned into the following types:

| <i><b>Node Type</b></i> | <i><b>Examples</b></i>              | <i><b>Description</b></i>                                                                                                                                                                                                                                                       |
|-------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Production              | Oil fields, extraction platforms    | Assets that generate crude oil. Key attributes: production rate, capacity, utilization rate.                                                                                                                                                                                    |
| Storage                 | Tank farms, strategic reserves      | Assets that hold inventory. Key attributes: current inventory level, storage capacity, target stock level (historical average).                                                                                                                                                 |
| Transit                 | Pipeline junctions, port terminals  | Assets that facilitate the transfer of flows between routes. Key attributes: throughput capacity, current throughput, transit time.                                                                                                                                             |
| Vessel                  | VLCC, Aframax, Suezmax tankers      | Maritime transport assets carrying cargo inventory during transit. Key attributes: cargo capacity, current cargo level, voyage status. Flow edges to all potential origin and destination terminals are permanent; flows are zero when the vessel is inactive or repositioning. |
| Refinery                | Refineries, processing plants       | Assets that transform crude into products. Key attributes: processing capacity, yield rates, current throughput.                                                                                                                                                                |
| Consumption             | Distribution terminals, end markets | Applicable mostly to distilled products, or in certain cases, when crude is used directly (i.e. production of electricity)                                                                                                                                                      |

An important design decision concerns the feature representation of nodes. One option is to define type-specific feature vectors where each node type carries only the variables relevant to its operational role — production rate only for production nodes, consumption rate only for consumption nodes. We adopt instead a uniform feature vector design in which every node, regardless of type, is characterised by the same set of time-varying variables. Specifically, each node  $i$  at time step  $t$  is described by: production rate  $P(i, t)$ , consumption rate  $C(i, t)$ , inventory level  $I(i, t)$ , total inflow  $F_{in}(i, t)$ , total outflow  $F_{out}(i, t)$ , and capacity utilisation as a fraction of maximum capacity. For variables that do not apply to a given node type, the value is simply zero — a storage node has  $P(i, t) = 0$  and  $C(i, t) = 0$ ; a vessel node has  $P(i, t) = 0$  and  $C(i, t) = 0$  with  $I(i, t)$  representing its current cargo level. Likewise, flow edges between asset pairs that are currently inactive carry zero-valued flow time series rather than being absent from the graph, preserving a stable topology across all time steps.

There are four reasons for this uniform representation. First, standard GNN architectures (GCN, GAT, GraphSAGE) require a fixed-dimensional input for every node; type-specific vectors of

different lengths would require padding, masking, or separate processing pipelines. Second, the pattern of zero and nonzero values implicitly encodes each node's functional role, allowing the GNN to learn node type distinctions from data rather than requiring explicit categorical encoding. Third, the uniform representation naturally accommodates hybrid nodes — assets that combine functions, such as a port terminal with both transit and storage capabilities — which would be awkward to classify under a rigid type-specific schema. Fourth, it ensures that the inventory conservation equation applies universally across all nodes with no special-case logic.

This design follows established practice in the spatio-temporal GNN literature. In traffic forecasting, DCRNN (Li et al., 2018) and STGCN (Yu et al., 2018) represent all sensor nodes with the same set of variables (speed, flow, occupancy), regardless of whether a sensor sits at a highway junction, an arterial road, or a residential street — functional differences show up in the values, not in the structure of the feature vector. In the supply-chain domain, Wasi et al. (2024) benchmarked both homogeneous and heterogeneous graph formulations on the SupplyGraph dataset and found that homogeneous GNN models performed competitively with heterogeneous ones while being substantially simpler to implement and train. The uniform approach offers a favourable balance of generality, scalability, and compatibility with the full range of vectorisation methods under consideration.

The uniform feature vector is extended for refinery nodes to address the limitation identified in Section 2.2.1. Beyond the time-varying operational variables, each refinery node carries a set of static configuration attributes drawn from publicly available sources — for the example we are applying the study on, the EIA, annual refinery capacity report and the Oil and Gas Journal refinery survey. These cover crude distillation capacity, which conversion units are present (FCC, hydrocracker, delayed coker, visbreaker), the Nelson Complexity Index, and the crude quality envelope the refinery can handle expressed as an API gravity range and maximum sulphur tolerance. A refinery without a hydrocracker cannot economically process Venezuelan extra-heavy crude regardless of price signals; one with a large FCC but no desulphurisation unit is constrained to low-sulphur feedstocks. Encoding these constraints as fixed node features allows the GNN to generalise across refineries with similar configurations even when their flow histories differ, and — critically — allows the model to represent potential feedstock transitions that have not yet occurred in the training window but are physically plausible given the refinery's equipment.

The same principle applies to vessel nodes. A VLCC node carries its own set of static physical attributes — loaded draught, beam, and DWT class — that define which ports it can enter, which canals it can transit, and what cargo volumes it can carry. A vessel with a 22-metre loaded draught cannot enter most conventional port berths without lightering; one with a beam exceeding 32.3 metres cannot transit the original Panama Canal locks. These constraints are not inferred from voyage history — they are fixed properties of the asset, available from Lloyds Register or the IHS Markit fleet database. Encoding them as static node features means that a vessel with permanent edges to West African, North Sea, and US Gulf Coast terminals carries, in its feature vector, the information the GNN needs to learn which of those routes are physically accessible given current freight economics — even if only one corridor has been observed active in the training data. The configuration-grounding principle is therefore not a special case for refineries: it is a general

property of the asset-centric framework, applicable wherever a node's physical characteristics define a feasible operating envelope that its flow history may underrepresent.

### 3.3 Edge Types and Attributes

Edges in the graph represent physical or contractual connections through which crude oil or products flow. The edge set  $E$  includes:

| Edge Type      | Connectivity                                   | Key Attributes                                                                |
|----------------|------------------------------------------------|-------------------------------------------------------------------------------|
| Pipeline       | Production → Storage, Storage → Refinery, etc. | Flow capacity, current flow rate, transit time, distance, operational status. |
| Pipeline       | Production → Storage, Storage → Refinery, etc. | Flow capacity, current flow rate, transit time, distance, operational status. |
| Shipping Route | Port → Port                                    | Vessel capacity, transit time, frequency, current utilization.                |
| Road/Rail      | Storage → Consumption, Refinery → Consumption  | Transport capacity, cost per unit, transit time.                              |

Each edge  $(i, j)$  carries a time-varying feature vector  $e(i, j, t)$  recording the flow rate  $f(i, j, t)$ , the transit time  $\tau(i, j)$ , and capacity utilisation. Edges are directed, reflecting the physical direction of flow, though some connections may be bidirectional — modelled as two directed edges.

### 3.4 Temporal Graph Formulation

The complete logistics network at time  $t$  is represented as a temporal graph  $G(t) = (V, E, X(t), W(t))$ , where  $X(t)$  is the matrix of node features and  $W(t)$  is the matrix of edge features at time  $t$ . Over the time horizon  $T = \{1, 2, \dots, T\}$ , the system is described by the sequence of graphs  $\{G(1), G(2), \dots, G(T)\}$ . In the general case, both the graph topology  $(V, E)$  and the feature matrices  $(X, W)$  may change over time; for the crude-oil logistics network at the regional level, however, the topology is relatively stable — pipelines and shipping routes do not change daily — so the primary source of temporal variation lies in the node and edge features.

The choice between the discrete-time approach — treating each  $G(t)$  as a separate graph snapshot — and the continuous-time approach — processing individual flow events as they occur — is one of the methodological questions addressed in Chapter 4.

### 3.5 Inventory Conservation as a Graph Constraint

The inventory conservation equation introduced in Chapter 2 takes a precise form within the graph schema. For each node  $i$  with storage capability, at each time step  $t$ :

$$I(i, t) = I(i, t-1) + P(i, t) + \sum_{j \in N_{in}(i)} f(j, i, t-\tau(j, i)) - C(i, t) - \sum_{k \in N_{out}(i)} f(i, k, t)$$

where  $N_{in}(i)$  and  $N_{out}(i)$  denote the sets of predecessor and successor nodes of  $i$  in the graph, and  $\tau(j, i)$  is the transit time on edge  $(j, i)$ . This equation couples the dynamics of node  $i$  to the

dynamics of all its neighbours, with the coupling mediated by edge flow rates and transit times. The graph structure  $G$  determines which nodes are coupled and through which edges, while the temporal dynamics determine how these couplings evolve.

Under the mean-reversion objective, the target state for each storage node  $i$  is  $I^*(i) = \bar{A}(I(i))$ , the historical average inventory. The forecasting task consists of predicting the flow adjustments  $\Delta f(i, j, t)$  that would drive the system from its current state toward this target, subject to the conservation equation and capacity constraints on all nodes and edges.

### 3.6 Mathematical Framework for Node and Edge Definitions

The graph schema defined in the preceding sections rests on a formal mathematical framework governing how nodes and edges are defined, how variables are assigned, and how the network's internal consistency is verified. Topology, semantics, and validity are kept separate: the graph is built from variable relationships first, and consistency is checked second as a post-build validation pass.

There are only two primitives: nodes, which own variables, and formulas, which create edges. The graph builder needs no knowledge of geography, administrative hierarchies, or reporting conventions — all that complexity is absorbed into the formulas and deferred to the validation pass. Adding new data sources, node types, commodity grades, or derived relationships requires only writing new formulas; the underlying graph machinery does not change.

Three concrete examples illustrate why this matters. Suppose a data source reports US crude oil production at the state level, but refinery runs (consumption) are only available at the PADD level — a coarser regional aggregation used by the EIA. A traditional schema with explicit granularity tiers would require a special aggregation layer and double-counting guards. Under the present framework, one simply writes:  $\text{PADD\_consumption}(p, t) = \text{sum of } C(i, t) \text{ for all states } i \text{ within PADD } p$ . The formula creates the edges automatically, and the mass balance check confirms whether the aggregation is internally consistent. No schema change is required.

Consider next an export terminal located within a state. The terminal is a pass-through node with no production and no consumption of its own; its sole function is routing outflows from the state to foreign destinations. Under the present framework, this is expressed as two formulas: the terminal's inflow equals the portion of the state's outflow routed via the terminal, and the terminal's outflow equals the sum of flows to all destination nodes. The terminal node appears in the graph with the correct connectivity, and the state-level mass balance automatically accounts for the volume that leaves via the terminal. No special sub-node logic or containment hierarchy is needed.

Third: if a future extension requires modelling refined products alongside crude, adding gasoil as a new commodity grade requires only introducing grade-indexed variable instances on the relevant nodes and writing a yield formula (e.g.,  $\text{gasoil\_production} = 0.5 \times \text{crude\_consumption}$  at a refinery node). Every other aspect of the framework — the mass balance constraint, the balancing item calculation, the GNN feature vectors, the validation logic — applies to the new grade without modification.

### 3.6.1 Node Definition

A node is defined minimally as a tuple  $i = (id, type)$ , where type encodes the operational role of the asset (production site, storage facility, terminal, refinery, consumption point, etc.). Location, administrative granularity, and other metadata are deliberately excluded from the node definition itself; they are retained as external attributes used only during the post-build validation phase.

Each node  $i$  carries a set of variables per commodity grade  $g$  and time step  $t$ . Following the uniform feature vector design from Section 3.2, these are: production  $P(i,g,t)$ , total inflow  $F_{in}(i,g,t)$ , consumption  $C(i,g,t)$ , total outflow  $F_{out}(i,g,t)$ , and stock level  $S(i,g,t)$ . Each variable is assigned one of two forms: a time series of observed data, or a formula expressed as a function of variables belonging to the same node or to other nodes. This distinction is fundamental to how the graph is constructed — it is the formulas that create the edges, not any explicit declaration of connectivity.

Each product or commodity grade is treated as a separate variable instance on the node, which supports derived-product relationships. At a refinery node, gasoil output can be defined as  $gasoil\_production(i,t) = 0.5 * C(i, crude, t)$ , where crude consumption drives refined product production. The formula creates an implicit dependency edge between the crude consumption variable and the gasoil production variable within the same node, and between connected nodes where cross-node references appear.

### 3.6.2 Edge Definition

An edge exists if and only if a variable on node  $i$  references a variable on node  $j$  through a formula. Formally,  $e = (i, j, g)$  exists if and only if there exists a variable  $v(i,g) = f(..., v(j,g), ...)$ . The graph topology is therefore fully determined by the variable formulas: edges are never declared separately, and a declared edge without a corresponding formula relationship cannot exist. This ensures the graph is always grounded in actual data dependencies, eliminating the possibility of structural inconsistency between the declared topology and the computational model.

Each edge  $e = (i, j, g)$  carries the flow volume  $\phi(i,j,g,t)$ , contributing symmetrically to both endpoints. The total outflow of node  $i$  equals the sum of flows on all edges departing  $i$ , and the total inflow of node  $j$  equals the sum of flows on all edges arriving at  $j$ :  $F_{out}(i,g,t) = \sum_j \phi(i,j,g,t)$  and  $F_{in}(j,g,t) = \sum_k \phi(k,j,g,t)$ . Additional edge attributes include transit time  $\tau(i,j)$  (relevant for tanker routes and long pipelines) and line loss  $\ell(i,j,g)$  (relevant for pipeline leakage and evaporation), which introduce a physical lag between when a flow departs node  $i$  and when it arrives at node  $j$ .

### 3.6.3 The Universal Mass Balance Constraint

Every node, regardless of type, must satisfy the mass balance equation. For node  $i$ , grade  $g$ , and time  $t$ :

$$\Delta S(i,g,t) = P(i,g,t) + F_{in}(i,g,t) - C(i,g,t) - F_{out}(i,g,t)$$

where  $\Delta S(i,g,t) = S(i,g,t) - S(i,g,t-1)$  is the observed change in stock. This equation holds regardless of whether the variables were assigned from time series or from formulas — it is the primary consistency check applied after the graph is built, not a constraint enforced during construction.



### 3.6.4 Stock, Balancing Item, and Temporal Resolution

For nodes that hold physical inventory — storage facilities, strategic reserves, pipeline segments with line-fill — the stock level  $S(i,g,t)$  is treated as a first-class time series variable populated from observed data. The change in stock  $\Delta S(i,g,t)$  implied by this series may not exactly equal the sum of the four flow variables  $P$ ,  $F_{in}$ ,  $C$ , and  $F_{out}$ , due to measurement error, unreported flows, rounding across data sources, and other data quality issues endemic to commodity reporting. The residual is captured by a dedicated variable termed the balancing item:

$$B(i,g,t) = \Delta S(i,g,t) - [ P(i,g,t) + F_{in}(i,g,t) - C(i,g,t) - F_{out}(i,g,t) ]$$

The term follows established practice in international energy statistics — the International Energy Agency uses it to absorb unaccounted differences in national energy supply-and-demand tables. The balancing item can be positive or negative, reflecting an apparent surplus or deficit in reported flows relative to observed stock change. Rather than treating it as noise to be discarded, this framework retains the balancing item as a node variable: it carries information about systematic data gaps that may themselves be predictable or structurally informative for the GNN. Whether the balancing item serves as a forecasting target, a regularisation signal, or a diagnostic flag is a modelling decision deferred to Chapter 5.

All time series variables are pre-processed to a common unit of barrels per day (b/d) before ingestion into the graph. Where source data report cumulative volumes over a period (monthly totals, weekly aggregates), these are converted to average daily rates by dividing by the number of days in the period. The resulting time series are treated as step functions: the reported rate for a given observation date persists unchanged until the next observation date, at which point it steps to the new value. This convention is consistent with standard practice in energy market reporting, where data providers publish period-average rates rather than instantaneous measurements, and it ensures that any two observations — regardless of their temporal spacing — can be compared and combined on a uniform basis without interpolation or frequency alignment in the graph layer.

### 3.6.5 Post-Build Consistency Validation

Once the graph has been constructed from its variable definitions, a set of consistency checks is applied using node type metadata and geographic location information. These checks verify physical plausibility rather than mathematical correctness, and are performed as a post-build validation pass rather than as construction constraints. Pass-through nodes (export terminals, pipeline segments) must have  $P = 0$  and  $C = 0$ ; production-only nodes (wellheads, extraction platforms) must have  $F_{in} = 0$  and  $C = 0$ ; edges between geographically disconnected nodes may flag a missing intermediate node or a data error; and formula-defined aggregation variables are cross-checked against the mass balance to verify that the aggregation is complete.

The separation of construction from validation is intentional. Mixed-granularity data — where some variables are reported at state level and others at sub-state or terminal level — requires no special handling in the construction phase. The formula layer handles granularity transitions naturally: if state-level exports are defined as the sum of all terminal outflows, that formula creates the correct edges automatically, and the consistency check confirms whether the



aggregation closes. No explicit granularity hierarchy or double-counting prevention logic is required in the schema.

The initial implementation uses crude oil without grade differentiation, simplifying the variable space while preserving the full structural and temporal complexity of the network. The schema is designed for extension: adding grades or refined products requires only introducing additional grade-indexed variable instances on each node, with no changes to the underlying graph construction or validation logic. Blending nodes — where multiple grades combine to produce a composite output — will be incorporated in future extensions by defining a blending contribution formula as a function of consumption, which integrates naturally into the existing formula-implies-edge framework.

## 4. Modelling the US Oil Network

Chapter 3 stated the schema. This chapter reports what the schema looks like once it is instantiated for the United States crude-oil network, the case study that motivated the framework. The presentation is compact by intent: scope, the resulting graph, the resolution choices made under the starter scope, the validation outcomes from the audit suite, and the artefacts the build produces. The full asset inventory, the data-source mapping and the routing detail are in Annex A.

### 4.1 Scope

The seeded model covers the contiguous United States, the offshore Gulf of America, Alaska and the Canadian crude inflow that lands in PADD 2 and PADD 3. Mexican imports are included where they affect PADD 3 routing. Refined-product distribution downstream of refineries is out of scope; the graph stops at the refinery transformation node. The temporal scope is 2015 to 2025, monthly, matching the EIA series that drive the starter scope binding.

### 4.2 The Resulting Graph

The starter graph has 214 nodes and 395 directed flow edges. By node class, 14 nodes are production, 25 are observational aggregates and 175 are infrastructure. The infrastructure category covers refineries, pipelines, storage and export terminals, gathering blocks, SPR sites and import terminals. Variables on the graph come to 1,646 in total, of which 856 are non-relational (production, consumption, inventory and balancing-item rows) and 790 are relational (the outflow rows that double as edges).

US production capacity in the curated metadata totals 11,690 kbd, matching the project-canonical 2023/24 average. The bulk of that total is concentrated in three plays: the Permian Basin (5,700 kbd), Eagle Ford (1,200 kbd) and Bakken (1,160 kbd). The offshore Gulf contributes 1,800 kbd, Alaska 420 kbd and the rest of the lower-48 conventional aggregate about 1,410 kbd. Canadian crude of about 3,700 kbd routes to the US through four cross-border pipelines, out of total Western Canadian Sedimentary Basin output of about 4,900 kbd. The named pipelines, refineries, terminals and SPR sites are listed in Annex A.

### 4.3 Resolution Choices Under the Starter Scope

The starter scope assigns one authoritative level per subsystem and treats the others as inert under that scope. The choice for each subsystem reflects the resolution at which EIA actually publishes data at monthly frequency. A finer-resolution scope is admissible without rebuilding the graph. If state-level production data later becomes available for the Permian or the Bakken, the corresponding scope flips the children to authoritative and the parent aggregate becomes a derived sum (Section 3.6.4).

| Subsystem         | Authoritative level       | Collapsed under starter scope     |
|-------------------|---------------------------|-----------------------------------|
| permian_subsystem | permian (basin aggregate) | permian_tx, permian_nm, gathering |

|                             |                     |                                   |
|-----------------------------|---------------------|-----------------------------------|
| <b>bakken_subsystem</b>     | bakken              | bakken_nd, bakken_mt, gathering   |
| <b>eagle_ford_subsystem</b> | eagle_ford_tx       | (none — single state)             |
| <b>alaska</b>               | alaska_north_slope  | ans_gathering                     |
| <b>gulf_of_america</b>      | gulf_of_america     | (none)                            |
| <b>rest_of_l48</b>          | rest_of_l48         | 4 state_conventional nodes        |
| <b>cushing_hub</b>          | cushing_hub         | 3 sub-terminals                   |
| <b>spr_subsystem</b>        | spr_sites (4 sites) | (none — site-level authoritative) |

## 4.4 Validation

Five audit scripts run against the loaded database after each rebuild and confirm the framework's invariants on the concrete graph.

**Coverage.** The audit confirms total US production capacity of 11,690 kbd against the project-canonical reference, broken down by basin and state aggregate. Canadian inflow is confirmed at about 3,700 kbd across the four cross-border pipelines plus a residual.

**Source-to-sink reachability.** Every production node reaches at least one refinery node along the directed flow projection, and every refinery node is reachable from at least one production node. There are no orphans.

**Mass balance.** Under the starter scope the audit reports zero residuals on every physical node. This is the intended state at the current binding density: most variables are unbound and the residual equation is vacuous. The audit's role is to fail loudly once bindings are added and the residuals exceed a configured threshold.

**Node typing.** Every node carries the variable complement its node\_type requires. Refineries carry production, consumption, inventory, balancing-item and one outflow per pipeline they feed. Basin aggregates carry the non-relational variables and no outflows.

**Routing regression.** The layered fixes applied during the build (the Permian sub-routing, the Cushing fan-out, the Houston export feed-in and the Canadian cross-border allocations) are confirmed against a checked-in expected-routing file.

## 4.5 Artefacts

Two interactive Plotly maps are produced alongside the loaded database.

`crude_logistics_map.html` plots the 214 nodes on a North America projection, with refinery hover tooltips carrying capacity, Nelson index and operator. `crude_logistics_edges.html` plots the 395 edges with hover-able bindings under the active scope. The schema diagrams (Figures A.1 and A.2 of Annex A) cover the data model and the asset → node → variable → assignment chain.

## 5. Vectorisation Readiness

This chapter argues that the schema developed in Chapter 3, and instantiated for the United States in Chapter 4, is fit for downstream machine learning. The argument takes the form of a fit analysis. Each schema decision is mapped to the input requirement it satisfies for at least one major family of graph vectorisation methods. The chapter is not a method survey; standard references on graph representation learning, graph convolutional networks, attention and temporal extensions cover that ground. What is offered here is the mapping itself, because the mapping makes explicit which schema invariants do which work for downstream machine learning, and which preprocessing burdens any concrete pipeline still has to handle.

### 5.1 Input Requirements of Graph Vectorisation Methods

A graph vectorisation method takes a graph (nodes, edges, features and, in the temporal case, timestamps) and returns a vector representation of nodes, edges, sub-graphs or whole graphs. The input requirements that recur across the major method families are fairly standard.

A method needs a deterministic adjacency, expressed as an edge list or sparse matrix without ambiguity about direction, multiplicity or self-loops. It needs node features that are either uniformly typed across the graph or labelled by node type so the method can condition on them. Edge features, where present, need to be typed and dimensioned consistently. The method needs a clear notion of when a feature value applies, and, for continuous-time methods, a way to read event timestamps. It needs the persistent topology of the graph held apart from the time-varying signals that ride on it. And it needs stable, dense node identifiers, since many methods use index-based parameter sharing.

The schema in Chapter 3 was not designed by going down that list. It was designed to make a multi-resolution logistics graph self-consistent, and the United States case study in Chapter 4 shows that the result is a graph that the standard pipelines can read. The claim of this chapter is that the principles that keep the schema self-consistent are also the principles that satisfy those vectorisation requirements, and that no further work is needed on the schema's part.

### 5.2 Mapping Schema Decisions to Vectorisation Requirements

The mapping from schema decisions to vectorisation requirements is the working element of this chapter. The discussion that follows walks through the principal entries.

Edges from variables. Because edges come out of relational variables on nodes, the adjacency is a deterministic projection of the variable catalogue. There is no separate edge table to drift out of step with the node table. Every edge is a tuple of source node, related node, variable type and commodity, and the projection is a single SELECT.

Aggregates as views. Aggregate nodes (basin views, PADD views) carry no outflow variables and therefore no edges. They show up as observation roll-ups, not flow hubs. Vectorisation methods that read the adjacency see the physical layer; methods that read node features see the aggregates through the same node-feature interface as the physical nodes, with the roll-up handled at the variable layer.

Per-scenario node scope. The asset graph and the variable catalogue are persistent. What changes per scenario is the binding of variables to time series, formulas or zero. Vectorisation methods that take a fixed topology and a sequence of feature snapshots see exactly that: one topology, many feature matrices.

Balancing items on every node. Each physical node carries a balancing-item variable that absorbs measurement residuals. Node feature vectors are therefore dense; there is no node for which the production / consumption / inventory / outflow row is incomplete. The balancing item is itself a feature, so methods that find it useful as a signal can use it.

Bidirectional pipelines as paired slots. A bidirectional pipeline is two outflow slots, one per direction, never one signed flow. Edge features stay non-negative, and direction lives in the adjacency rather than in the feature.

Hierarchy as a derived view. Parent-child relationships are not stored on their own table; they are derived from `variable_assignments` rows where the formula is `sum_over_children`. The hierarchy can be re-derived under any scope without rebuilding tables. Methods that benefit from hierarchical pooling read the hierarchy as a SQL query.

Stable, dense identifiers. Every node has a stable string identifier that is reused across scenarios and across rebuilds. Index assignment for machine learning is a single ordering pass over the nodes table.

Monthly temporal alignment. The starter scope binds all time-varying variables to monthly EIA series. Where weekly or daily series exist, they are downsampled to monthly at ingest. Continuous-time methods that prefer event timestamps can read the raw catalogue instead, since the data loader schema preserves the original frequency.

### 5.3 Residual Preprocessing Burden by Method Family

The mapping above shows structural fit. It does not yet say what preprocessing a downstream pipeline still has to do. This section walks through three representative vectorisation families and names the burden each one carries.

Structural embedding methods, such as `node2vec`, `DeepWalk` and `LINE`, treat the graph as a static topology and learn embeddings from random walks or local neighbourhood statistics. The schema satisfies their requirements directly. The adjacency is deterministic, node identifiers are stable, and the topology is persistent across scenarios. These methods do not consume node features at all, so node-feature heterogeneity is not an issue for them. They will, however, ignore the balancing item, the variable types and the multi-resolution scope information. Their use case in this thesis is therefore as a baseline that captures topology only.

Spatial GNN methods, such as `GCN`, `GAT` and `GraphSAGE`, consume both adjacency and node features. Beyond the structural fit established above, the burden a downstream pipeline has to handle is normalisation: node features need to be brought onto a common scale, and missing values need to be either masked out or interpreted as informative zeros. The schema makes the masking trivial, since the `assignment_kind` column already distinguishes a zero binding from a time-series binding from a formula binding. Methods that benefit from heterogeneous node- type information (`R-GCN`, `HGT`) read it from the `node_type` column.

Temporal GNN methods, such as TGAT, TGN and the various hybrid graph-plus-sequence architectures, need either an event-time interface or a snapshot sequence. The schema supports both. For snapshot-sequence methods, a query returns one feature matrix per period using the same persistent topology. For continuous-time methods, the `timeseries_data` table carries the original event timestamps, so the input becomes a stream of (node id, variable id, timestamp, value) tuples without further preprocessing.

Three burdens are not handled by the schema, and a downstream pipeline has to handle them itself. Methods that need positional encodings such as Time2Vec have to be told the period boundaries; the scenario row records the frequency but does not impose any particular encoding. Cross-scenario transfer learning is the user's problem; each scenario is independent, and joint training across scenarios with different authoritative levels has to be handled outside the schema. Methods that use a learnable adjacency commit to the asset graph as a hard prior; the framework gives no facility to soften it. None of these three is a property of the schema. Each is local to a particular method family, and each is named here so that the overall claim of fit is clear about what it covers and what it does not.

## 5.4 Summary

The argument here does not depend on any particular vectorisation method being the right one for the eventual forecasting study. It depends on the schema being a sound representational baseline for any of them. The mapping in Section 5.2 shows that the recurrent input requirements of graph vectorisation methods are met by the schema invariants developed in Chapter 3. The walk-through in Section 5.3 shows that the residual preprocessing burden is local to specific method families and does not call for schema changes. Chapter 6 returns to which method family the forecasting study should commit to, framed as future work.

## 6. Future Work: Forecasting Applications

The application that motivated the framework was forecasting crude-oil inventories under a mean-reversion objective. Several schema decisions trace back to that target application, including balancing-item retention, per-scenario binding and monthly temporal alignment. The forecasting study itself is left to subsequent research. This chapter records the roadmap so the hand-off is clean.

The chapter is written as future work, not as results. It states the forecasting problem in the schema's terms, names the candidate model families, describes the training and evaluation protocol the schema enables, and sketches a model selection strategy. Empirical results, ablations and final model choice are out of scope here.

### 6.1 Forecasting Problem in Schema Terms

The forecasting problem, expressed in the schema's terms, is to predict the inventory variable on a chosen set of physical nodes (the forecast targets) at horizons of one to twelve months ahead, given the multivariate state of the graph at and before the forecast issue date.

The forecast targets are the inventory-bearing nodes for which EIA publishes weekly or monthly stock series: Cushing, the Gulf Coast storage hubs and the four SPR sites. Other inventory variables in the graph, such as line and lease tankage on production nodes, are typically zero or unbound under the starter scope and are not direct forecast targets, though they may serve as auxiliary features.

The mean-reversion objective formalises a prior, namely that inventories tend to return to their typical operating range over multi-month horizons absent persistent shocks. The objective penalises forecasts that stay far from the rolling mean of the target series for longer than the historical envelope justifies. The exact functional form is a question for the forecasting study; the schema is agnostic to it.

### 6.2 Candidate Model Families

The candidate model families are spatial GNNs on snapshot sequences (GCN, GAT, GraphSAGE), temporal GNNs (TGN, TGAT), hybrid graph-plus- sequence architectures (graph + LSTM, graph + temporal convolution, Time2Vec encodings), and classical time-series baselines (ARIMA, VAR) for comparison. Choosing among them is the open question the forecasting study would answer. The schema does not prejudge it. Each family consumes the same underlying tables, with method-specific preprocessing as discussed in Section 5.3.

### 6.3 Training Strategy

The training strategy is constrained by the data. The seeded scope covers 2015 to 2025 monthly, giving 132 monthly observations per target. A train / validation / test split of 2015 to 2021 / 2022 to 2023 / 2024 to 2025 leaves enough data for hyper-parameter tuning while preserving an out-of-sample window. Cross-scenario training is admissible without schema changes if higher-frequency or finer- resolution scopes are added later.

## 6.4 Evaluation Metrics

Evaluation metrics should reflect the mean-reversion objective. RMSE on the forecast horizon is part of the picture, but so is calibration of the forecast distribution against the historical inventory envelope. Probabilistic forecasts (quantile or distributional) are appropriate, since point forecasts alone do not capture the asymmetric cost of inventory misses near operational limits.

## 6.5 Model Selection Strategy

Model selection should weigh three things: predictive accuracy on the held-out window, calibration to the historical envelope, and ablation evidence that the graph structure improves on classical baselines. The third criterion is the only one that justifies the graph framework against the simpler alternative of per-node ARIMA. A negative ablation result is acceptable as a thesis outcome; the schema retains its value as a representation even if the forecasting application benefits more from sequence structure than from graph structure.



## 7. Conclusion

This thesis defines and implements an asset-centric temporal graph schema for crude-oil logistics. The schema is built to be scalable and to be ready for downstream machine learning, both taken in their simple senses, and is applied to the United States crude-oil network as a working case study.

Scalability has two aspects, and both are met. Adding a new asset is a small edit to a handful of tables, with topology following from the variable catalogue and the audit suite catching mistakes. The 214-node starter graph was assembled in layers (the refinery layer, the residual layer, the Canadian layer, the inter-PADD flow edges, the realistic feeders), and each layer was a contained edit rather than a rebuild. On the observational side, the same graph supports any data resolution that public series happen to admit. Re-scoping from basin level to PADD level is an edit to the per-scenario binding rows, not to the asset graph.

The argument for machine-learning readiness is made structurally. Each schema decision is mapped to an input requirement of the standard graph vectorisation pipelines, and the residual preprocessing burden is named per method family. The mapping is what allows a forecasting study to begin from a sound representation rather than to have to rebuild one.

The case study confirms the framework on a real network. The starter graph closes US refining and production capacity to the project- canonical totals, the source-to-sink reachability matrix has no orphans, and the audit suite passes against the loaded database. Annex A documents the build end-to-end.

The contribution is therefore representational. It is the joint enforcement of a small set of design principles (edges from variables, aggregates as views, balancing items on every node, per-scenario node scope, paired slots for bidirectional pipelines, hierarchy as a derived view) as schema invariants, exposed in a working Postgres implementation that can be inspected, audited and extended.

The framework is open in two directions. Toward the data side, adding a new source (Canada Energy Regulator pipeline flows, Alberta production by play) is a one-loader-notebook change and a per-scenario re-binding. Toward the modelling side, the schema commits to no particular machine-learning method; the choice of forecasting architecture remains open.

## References

- Ahn, H., Song, Y. C., Olivar, S., Luo, S., and Ceballos, H. (2024). GNN-based Probabilistic Supply and Inventory Predictions in Supply Chain Networks. arXiv preprint arXiv:2404.07523.
- Alves, J. C. and Mateus, G. R. (2020). Deep Reinforcement Learning and Optimization Approach for Multi-Echelon Supply Chain with Uncertain Demands. In International Conference on Computational Logistics, pp. 584–599. Springer.
- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., and Ljung, G. M. (2015). Time Series Analysis: Forecasting and Control. 5th ed. Wiley.
- Dixit, A. K. and Pindyck, R. S. (1994). Investment Under Uncertainty. Princeton University Press.
- Dong, T. and Samonte, M. J. C. (2024). Optimization and Benefit Assessment of Production Supply Chain Networks Using Graph Neural Network Models. Journal of Computing and Information Technology, 32(1), pp. 15–31.
- Fattouh, B. (2011). An Anatomy of the Crude Oil Pricing System. Oxford Institute for Energy Studies, WPM 40. Oxford: University of Oxford.
- Fernandes, L. J., Relvas, S., and Barbosa-Póvoa, A. P. (2013). Strategic Network Design of Downstream Petroleum Supply Chains: Single versus Multi-Entity Participation. Chemical Engineering Research and Design, 91(9), pp. 1557–1587.
- Ghaithan, A. M., Attia, A., and Duffuaa, S. O. (2017). Multi-Objective Optimization Model for a Downstream Oil and Gas Supply Chain. Applied Mathematical Modelling, 52, pp. 689–708.
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. (2017). Neural Message Passing for Quantum Chemistry. Proceedings of the 34th International Conference on Machine Learning (ICML).
- Geman, H. (2005). Commodities and Commodity Derivatives: Modelling and Pricing for Agriculturals, Metals and Energy. Wiley.
- Grover, A. and Leskovec, J. (2016). node2vec: Scalable Feature Learning for Networks. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855–864.
- Hamilton, J. D. (1994). Time Series Analysis. Princeton University Press.
- Hamilton, W. L., Ying, Z., and Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. Advances in Neural Information Processing Systems (NeurIPS), 30.
- Jin, M., Koh, H. Y., Wen, Q., Zambon, D., Alippi, C., Webb, G. I., King, I., and Pan, S. (2024). A Survey on Graph Neural Networks for Time Series: Forecasting, Classification, Imputation, and Anomaly Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence.
- Kazemi, S. M., Goel, R., Eghbali, S., Ramanan, J., Sahota, J., Thakur, S., Wu, S., Smyth, C., Poupart, P., and Brubaker, M. (2019). Time2Vec: Learning a General-Purpose Representation of Time. arXiv preprint arXiv:1907.05321.

- Kazemi, Y. and Szmerekovsky, J. (2015). Modeling Downstream Petroleum Supply Chain: The Importance of Multi-Mode Transportation to Strategic Planning. *Transportation Research Part E: Logistics and Transportation Review*, 83, pp. 111–125.
- Kilian, L. and Murphy, D. P. (2014). The Role of Inventories and Speculative Trading in the Global Market for Crude Oil. *Journal of Applied Econometrics*, 29(3), pp. 454–478.
- Kipf, T. N. and Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.
- Kosasih, E. E. and Brintrup, A. (2022). A Machine Learning Approach for Predicting Hidden Links in Supply Chain with Graph Neural Networks. *International Journal of Production Research*, 60(17), pp. 5380–5393.
- Li, Y., Yu, R., Shahabi, C., and Liu, Y. (2018). Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.
- Lima, C., Relvas, S., and Barbosa-Póvoa, A. P. (2016). Downstream Oil Supply Chain Management: A Critical Review and Future Directions. *Computers and Chemical Engineering*, 92, pp. 78–92.
- Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T., and Leiserson, C. (2020). EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04), pp. 5363–5370.
- Perozzi, B., Al-Rfou, R., and Skiena, S. (2014). DeepWalk: Online Learning of Social Representations. *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 701–710.
- Ribas, G. P., Hamacher, S., and Street, A. (2010). Optimization Under Uncertainty of the Integrated Oil Supply Chain Using Stochastic and Robust Programming. *International Transactions in Operational Research*, 17(6), pp. 777–796.
- Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Bronstein, M., and Monti, F. (2020). Temporal Graph Networks for Deep Learning on Dynamic Graphs. *ICML 2020 Workshop on Graph Representation Learning*.
- Stopford, M. (2009). *Maritime Economics*. 3rd ed. London: Routledge.
- Schwartz, E. S. (1997). The Stochastic Behavior of Commodity Prices: Implications for Valuation and Hedging. *The Journal of Finance*, 52(3), pp. 923–973.
- Tang, J., Qu, M., Wang, M., Zhang, M., Yan, J., and Mei, Q. (2015). LINE: Large-Scale Information Network Embedding. *Proceedings of the 24th International Conference on World Wide Web (WWW)*, pp. 1067–1077.
- Uhlenbeck, G. E. and Ornstein, L. S. (1930). On the Theory of the Brownian Motion. *Physical Review*, 36(5), pp. 823–841.
- Vasicek, O. (1977). An Equilibrium Characterization of the Term Structure. *Journal of Financial Economics*, 5(2), pp. 177–188.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph Attention Networks. Proceedings of the 6th International Conference on Learning Representations (ICLR).

Wang, Y., Chang, Y., Li, Y., and Tan, K.-L. (2021). Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks. Proceedings of the 9th International Conference on Learning Representations (ICLR).

Wasi, A. T., Islam, S. R., Karim, R., and Brintrup, A. (2024). SupplyGraph: A Benchmark Dataset for Supply Chain Planning using Graph Neural Networks. arXiv preprint arXiv:2401.15299.

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., and Yu, P. S. (2020). A Comprehensive Survey on Graph Neural Networks. IEEE Transactions on Neural Networks and Learning Systems, 32(1), pp. 4–24.

Xu, D., Rong, Y., Huang, J., Zhu, W., and Leskovec, J. (2020). Inductive Representation Learning on Temporal Graphs. Proceedings of the 8th International Conference on Learning Representations (ICLR).

Yu, B., Yin, H., and Zhu, Z. (2018). Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI), pp. 3634–3640.